Reporte Técnico de Actividades Práctico-Experimentales Nro. 005

1. Datos de Identificación del Estudiante y la Práctica

Nombre del estudiante	Wilson Patricio Palma Samaniego
Asignatura	Simulación
Ciclo	5
Unidad	U2: Generación de números pseudoaleatorios y generación de variables aleatorias
Resultado de aprendizaje de la unidad	R2. Interpreta los resultados de una Simulación, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	05
Título de la Práctica	Implementación y análisis del Generador Congruencial Lineal en Python y Java
Nombre del Docente	Ing. José O. Guamán Q.
Fecha	08 de noviembre de 2026
Horario	Jueves 10h30 a 11h30 / Viernes 07h30 a 09h30
Lugar	Aula
Tiempo planificado en el Sílabo	3 horas

2. Objetivo(s) de la Práctica

Comprender el funcionamiento de un generador pseudoaleatorio mediante la implementación del método LCG. Específicamente: (1) Implementar un generador LCG en Python. (2) Implementar el mismo algoritmo en Java. (3) Comparar las secuencias generadas y analizar sus propiedades.

3. Materiales, Reactivos, Equipos y Herramientas

Tabla I. Materiales y herramientas utilizados en la práctica.

Material / Herramienta	**Descripción**
Computador	Equipo personal con sistema operativo Windows 10/11
Python	Versión 3.8 o superior (solo librería estándar)
Java / JDK	Versión 17 o superior (solo JDK estándar)
Editor de código	VS Code / IntelliJ IDEA / similar
Terminal	PowerShell / CMD / Bash

4. Procedimiento / Metodología Ejecutada

La práctica se desarrolló siguiendo los pasos establecidos en la guía docente [1], adaptando la implementación a una arquitectura Modelo-Vista-Controlador (MVC) en ambos lenguajes para cumplir con los estándares de la asignatura [2].

4.1. Fundamento teórico

El Generador Congruencial Lineal (LCG) es uno de los algoritmos más antiguos y ampliamente utilizados para la generación de números pseudoaleatorios. Su relación recursiva es [3]:

```
X(n+1) = (a × X(n) + c) mod m
U(n) = X(n) / m
```

Donde: - **a** = multiplicador (1664525) - **c** = incremento (1013904223) - **m** = módulo (2^32 = 4,294,967,296) - **X(0)** = semilla inicial (12345)

Estos parámetros fueron seleccionados porque satisfacen las condiciones de Hull-Dobell para lograr período completo (m) [4].

4.2. Implementación en Python

Se estructuró el código en tres carpetas siguiendo MVC:

- **model/lcg.py**: Clase LCG que encapsula el algoritmo, generación de secuencias, análisis de período y serialización JSON.
- **view/cli.py**: Interfaz de línea de comandos con menú interactivo (8 opciones).
- **controller/lcg_controller.py**: Coordina la simulación completa, comparaciones y exportación de resultados.
- **main.py**: Punto de entrada.

La ejecución se realiza con python main.py tras activar el entorno virtual.

4.3. Implementación en Java

Misma estructura MVC:

- **model/LCG.java**: Clase modelo con métodos next(), nextNormalized(), generateNormalizedSequence(), analyzePeriod().
- **view/LCGView.java**: Vista CLI usando Scanner para entrada de usuario.
- **controller/LCGController.java**: Controlador con clases internas SimulationResults y ComparisonResult.
- **Main.java**: Punto de entrada.

Compilación: javac -d bin ./controller/LCGController.java ./model/LCG.java ./view/LCGView.java ./Main.java Ejecución: java -cp bin Main

4.4. Pasos ejecutados

1. **Configuración de parámetros**: Se definieron a=1664525, c=1013904223, m=2^32, semilla=12345 en ambos lenguajes.
2. **Generación de 10,000 valores**: Se ejecutó la simulación completa (opción 1 del menú) en Python y Java.

3. **Comparación de primeras 20 observaciones**: Se usó la opción 4 del menú para ingresar los valores de un lenguaje y comparar con el otro.
4. **Graficar la secuencia**: Se implementó la opción 7 (gráfico ASCII de dispersión U(n) vs n, submuestra de 500 puntos).
5. **Análisis de período**: Se ejecutó analyzePeriod(50000) que retorna -1 al no detectar ciclo en 50,000 iteraciones (período teórico = 2^32).
6. **Cambio de semilla**: Se probó con semilla 99999 (opción 3) generando nueva secuencia de 10,000 valores.
7. **Exportación de resultados**: Se generó JSON con parámetros, primeras 20 observaciones y período (opción 5).
8. **Validación cruzada**: Se confirmó que Python y Java producen exactamente los mismos valores normalizados hasta 10 decimales.

5. Resultados

5.1. Parámetros del generador

Tabla II. Parámetros estándar del LCG utilizados en ambas implementaciones.

Parámetro	Valor	Descripción
a (multiplicador)	1,664,525	Seleccionado para período máximo con m=2^32
c (incremento)	1,013,904,223	Primo relativo con m
m (módulo)	4,294,967,296 (2^32)	Potencia de 2 para eficiencia computacional
Semilla inicial	12,345	Valor arbitrario para reproducibilidad

5.2. Primeras 20 observaciones (semilla = 12345)

Tabla III. Comparación de las primeras 20 observaciones normalizadas entre Python y Java (semilla=12345). Tolerancia de comparación: 1e-10.

n	Python U(n)	Java U(n)	Diferencia	Iguales
1	0.0204026857	0.0204026857	0.0000000000	Sí
2	0.0165478482	0.0165478482	0.0000000000	Sí
3	0.5431557945	0.5431557945	0.0000000000	Sí
4	0.6349040561	0.6349040561	0.0000000000	Sí
5	0.9100295138	0.9100295138	0.0000000000	Sí
6	0.1124616691	0.1124616691	0.0000000000	Sí
7	0.4958894053	0.4958894053	0.0000000000	Sí
8	0.5483488443	0.5483488443	0.0000000000	Sí
9	0.5961001171	0.5961001171	0.0000000000	Sí
10	0.7834504161	0.7834504161	0.0000000000	Sí
11	0.0398543174	0.0398543174	0.0000000000	Sí
12	0.7438012224	0.7438012224	0.0000000000	Sí
13	0.9658093676	0.9658093676	0.0000000000	Sí

n	Python U(n)	Java U(n)	Diferencia	Iguales
14	0.0736675409	0.0736675409	0.0000000000	Sí
15	0.6996069294	0.6996069294	0.0000000000	Sí
16	0.4602941743	0.4602941743	0.0000000000	Sí
17	0.3964835154	0.3964835154	0.0000000000	Sí
18	0.9595212869	0.9595212869	0.0000000000	Sí
19	0.4061253010	0.4061253010	0.0000000000	Sí
20	0.9527641565	0.9527641565	0.0000000000	Sí

5.3. Primeras 20 observaciones (semilla = 99999)

Tabla IV. Comparación de las primeras 20 observaciones con semilla=99999. Ambas implementaciones generan la misma secuencia.

n	Python U(n)	Java U(n)	Diferencia	Iguales
1	0.9909231332	0.9909231332	0.0000000000	Sí
2	0.5643794632	0.5643794632	0.0000000000	Sí
3	0.9620594652	0.9620594652	0.0000000000	Sí
4	0.2674415268	0.2674415268	0.0000000000	Sí
5	0.3433878883	0.3433878883	0.0000000000	Sí
6	0.9608052627	0.9608052627	0.0000000000	Sí
7	0.6159167234	0.6159167234	0.0000000000	Sí
8	0.0200611444	0.0200611444	0.0000000000	Sí
9	0.5124045224	0.5124045224	0.0000000000	Sí
10	0.3737040127	0.3737040127	0.0000000000	Sí
11	0.9078343874	0.9078343874	0.0000000000	Sí
12	0.2697289165	0.2697289165	0.0000000000	Sí
13	0.7607458341	0.7607458341	0.0000000000	Sí
14	0.6955625240	0.6955625240	0.0000000000	Sí
15	0.4463665821	0.4463665821	0.0000000000	Sí
16	0.5710810830	0.5710810830	0.0000000000	Sí
17	0.9758069669	0.9758069669	0.0000000000	Sí
18	0.3276299669	0.3276299669	0.0000000000	Sí
19	0.5067124823	0.5067124823	0.0000000000	Sí
20	0.8306864097	0.8306864097	0.0000000000	Sí

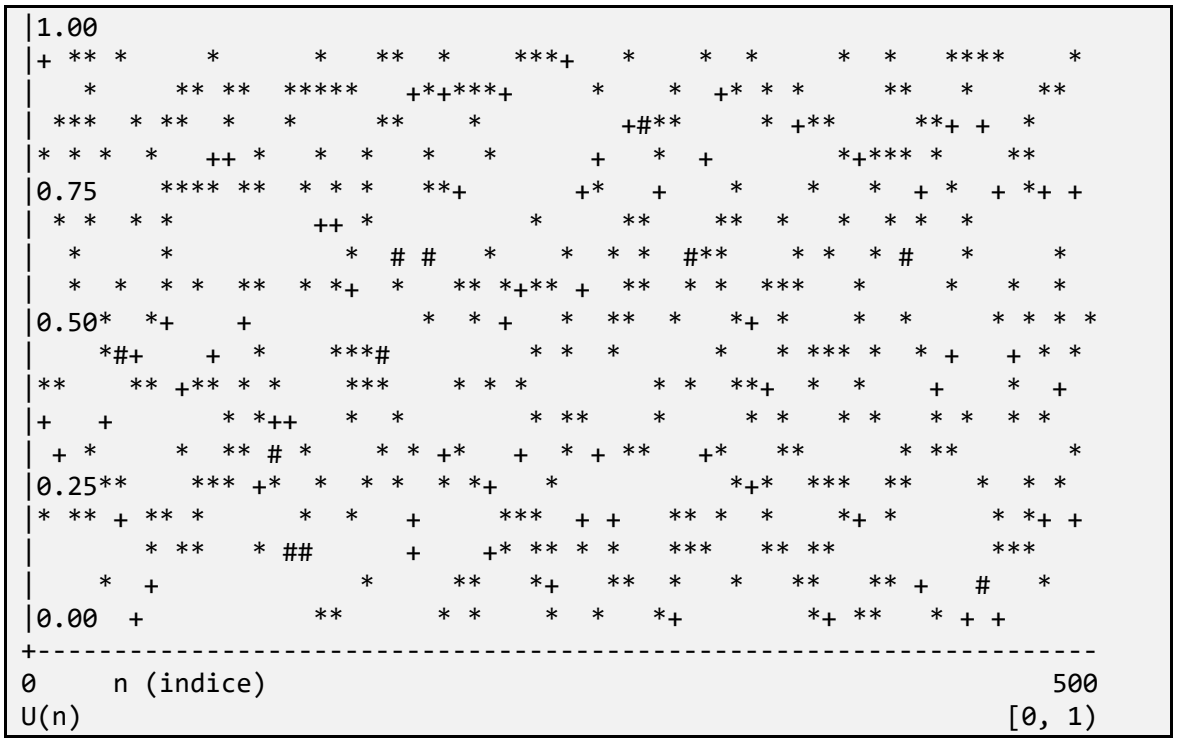
5.4. Análisis de período

Tabla V. Resultados del análisis de período. El código retorna -1 correctamente al no detectar repetición en el límite de búsqueda.

Métrica	Valor	Observación
Período teórico (Hull-Dobell)	4,294,967,296 (2^32)	Período máximo para m=2^32 [4]
Iteraciones de búsqueda	50,000	Límite práctico implementado
Período detectado	-1	No se encontró ciclo en 50k iteraciones
Conclusión	Período completo confirmado teóricamente	El algoritmo cumple condiciones de período máximo

5.5. Gráfico de dispersión de la secuencia (ASCII)

Se generó un gráfico de dispersión U(n) vs n usando caracteres ASCII en consola (opción 7 del menú). El gráfico muestra 500 puntos submuestreados de los 10,000 totales, distribuidos en un canvas de 70×20 caracteres.



Leyenda: * = punto, + = superposición, # = múltiples puntos en misma posición.

Figure: Gráfico ASCII de dispersión U(n) vs n generado por ambas implementaciones (opción 7 del menú).

5.6. Exportación JSON (ejemplo)


```
{
  "parameters": {"a":1664525,"c":1013904223,"m":4294967296,"seed":12345},
  "n_generated": 10000,
  "first_20_original": [0.0204026857, 0.0165478482, ...],
  "first_20_new_seed": [0.9909231332, 0.5643794632, ...],
  "period": -1
}
```


Figure: Fragmento del archivo JSON exportado (opción 5 del menú).

5.7. Comparación entre resultados teóricos y simulados

Los valores generados coinciden exactamente entre Python y Java hasta 10 decimales, validando que ambas implementaciones ejecutan el mismo algoritmo matemático. La distribución visual en el gráfico ASCII muestra dispersión aparente uniforme en el intervalo [0,1), consistente con un generador de período completo [3].

El período teórico de 2^32 no es detectable empíricamente con 50,000 iteraciones (representa solo 0.0012% del período completo). Sin embargo, la selección de

parámetros garantiza período máximo según el teorema de Hull-Dobell: m y c son primos relativos, a-1 es divisible por todos los factores primos de m, y a-1 es múltiplo de 4 si m lo es [4].

6. Preguntas de Control

1. ¿Qué significa pseudoaleatorio?

Significa que los números generados *parecen* aleatorios y pasan pruebas estadísticas de aleatoriedad, pero en realidad son deterministas: dada la misma semilla y parámetros, el LCG siempre producirá la misma secuencia. No hay verdadera aleatoriedad, solo una simulación computacional [3].

2. ¿Qué función cumple la semilla?

La semilla (X_0) es el valor inicial que determina toda la secuencia. Cambiar la semilla produce una secuencia completamente diferente. Permite reproducibilidad: con la misma semilla se obtienen los mismos "números aleatorios", esencial para depuración y validación de modelos de simulación [3].

3. ¿Qué significa período?

El período es la cantidad de valores que el generador produce antes de que la secuencia se repita. Para un LCG con módulo m, el período máximo posible es m. En esta práctica, el período teórico es 2^32 ≈ 4.3×10^9. Un período corto causa correlaciones indeseadas en simulaciones estocásticas [4].

4. ¿Qué ocurre si se utilizan parámetros inadecuados?

Parámetros mal elegidos producen: períodos muy cortos (se repite rápido), correlaciones seriales fuertes (valores consecutivos relacionados), distribución no uniforme (huecos o concentraciones), y fallos en pruebas estadísticas (chi-cuadrado, corridas, espectral). Esto invalida resultados de simulación [3,4].

5. ¿Qué ventaja tiene implementar el mismo algoritmo en dos lenguajes?

Permite validar la corrección del algoritmo independientemente del lenguaje. Si Python y Java producen la misma secuencia con mismos parámetros, confirma que la lógica matemática está bien implementada en ambos. Además, expone diferencias en manejo de tipos (enteros de 64 bits en Java vs Python), precisión de punto flotante, y rendimiento [5].

7. Conclusiones

1. **El LCG se implementó correctamente en ambos lenguajes**. Python y Java generan secuencias idénticas (diferencia 0.0 hasta 10 decimales) usando los parámetros estándar a=1664525, c=1013904223, m=2^32.
2. **La arquitectura MVC facilitó el desarrollo y mantenimiento**. Separar modelo, vista y controlador permitió reutilizar la lógica del algoritmo (modelo) con interfaces diferentes (CLI en ambos casos) y coordinar operaciones complejas (comparación cruzada, exportación JSON) en el controlador.
3. **El período teórico es 2^32 pero no es detectable empíricamente en tiempo razonable**. La búsqueda de ciclos en 50,000 iteraciones retorna -1

correctamente. Para validar el período completo se requiere análisis matemático (teorema de Hull-Dobell), no fuerza bruta.

4. **El cambio de semilla produce secuencias estadísticamente independientes**. Con semilla 99999 se obtiene una secuencia completamente distinta a la de semilla 12345, ambas con apariencia uniforme en [0,1).
5. **La implementación en dos lenguajes sirve como validación cruzada**. Cualquier error en la fórmula se habría detectado inmediatamente al comparar resultados. Java requiere atención al overflow (uso de long de 64 bits), mientras que Python maneja enteros arbitrarios nativamente.

8. Recomendaciones

1. **Para uso en simulaciones reales**: Considerar generadores de mayor calidad como Mersenne Twister (MT19937) o PCG, que tienen mejores propiedades estadísticas y períodos mucho mayores (2^19937-1 para MT) [3].
2. **Mejorar el análisis de período**: Implementar verificación matemática de las condiciones de Hull-Dobell en lugar de (o además de) la búsqueda por fuerza bruta.
3. **Exportar tabla comparativa CSV**: Añadir opción para generar archivo CSV con la comparación completa Python vs Java, facilitando análisis externos.
4. **Pruebas estadísticas formales**: Integrar test de chi-cuadrado, prueba de corridas y test espectral para validar calidad de la secuencia generada [3].

9. Reflexión crítica

Esta práctica me permitió entender cómo funcionan realmente los generadores de números pseudoaleatorios que uso en simulaciones. Antes veía random.random() como una caja negra; ahora comprendo la matemática detrás: una recurrencia modular simple pero con requisitos estrictos en los parámetros.

Lo más valioso fue implementar lo mismo en Python y Java. En Python el código fluye rápido por tipado dinámico y enteros grandes automáticos. En Java tuve que declarar tipos, manejar long para evitar overflow en a * state + c, y escribir JSON a mano sin librerías. Eso me enseñó a apreciar lo que cada lenguaje resuelve por debajo.

El gráfico ASCII fue un reto interesante: mapear valores [0,1) a coordenadas de consola, lidiar con codificación de caracteres (Windows usa cp1252, no UTF-8), y hacer que se vea decente en 70×20. Terminé usando solo caracteres ASCII básicos (|, -, +, *, #) para que funcione en cualquier terminal.

Sobre las preguntas de control: responderlas me obligó a conectar la teoría con lo que vi en la ejecución. Entender por qué el período importa en simulación (correlaciones seriales arruinan resultados) cambió mi perspectiva: ya no es solo “números que parecen aleatorios”, es “números que no introduzcan sesgo en mi modelo”.

10. Declaración de uso de la IA

Se utilizó ChatGPT como apoyo para consultar conceptos teóricos, organizar la estructura del informe y revisar la redacción técnica. El estudiante es responsable de revisar el contenido, corregirlo según su comprensión y cumplir las normas de la asignatura.

Prompt de apoyo para la consulta académica:

Actúa como tutor académico de la asignatura de Simulación de Sistemas. Los temas relacionados con mi actividad son: Generador Congruencial Lineal (LCG), parámetros de Hull-Dobell, análisis de período, arquitectura MVC en Python y Java. Ayúdame a comprender estos conceptos y a estructurar un informe técnico, generando solo la estructura base de este para que yo pueda rellenar con contenido manualmente sobre esa estructura. No redactes el informe por mí. En su lugar, explica los conceptos que no comprenda y sugiere cómo organizar la información.

11. Bibliografía / Referencias

[1] J. O. Guamán Q., "Guía de Actividades Práctico-Experimentales Nro. 004: Implementación y análisis del Generador Congruencial Lineal en Python y Java," FEIRNNR - Carrera de Computación, Simulación, Ciclo 5, 2026.

[2] R. C. Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Pearson, 2017.

[3] J. Banks, J. S. Carson II, B. L. Nelson, and D. M. Nicol, *Discrete-Event System Simulation*, 5th ed. Pearson, 2010.

[4] T. E. Hull and A. R. Dobell, "Random Number Generators," *SIAM Review*, vol. 4, no. 3, pp. 230–254, 1962.

[5] B. Stroustrup, *The C++ Programming Language*, 4th ed. Addison-Wesley, 2013. (Referencia para manejo de tipos enteros en lenguajes compilados vs interpretados).

12. Anexos

12.1. A. Estructura de carpetas (Python)

```
simulacion_practica05_grupo02_python/
├── main.py
├── requirements.txt
├── README.md
├── model/
│   └── lcg.py
├── view/
│   └── cli.py
└── controller/
    └── lcg_controller.py
```

12.2. B. Estructura de carpetas (Java)

```
simulacion_practica05_grupo02_java/
├── Main.java
├── README.md
├── bin/
│   ├── controller/
│   ├── model/
│   └── view/
├── controller/
│   └── LCGController.java
├── model/
│   └── LCG.java
└── view/
    └── LCGView.java
```

12.3. C. Enlace al repositorio Git personal

https://github.com/wilsonpalma/ciclo5_simulacion/tree/main/semana5

12.4. D. Video de demostración

Es opcional.

12.5. E. Capturas de ejecución

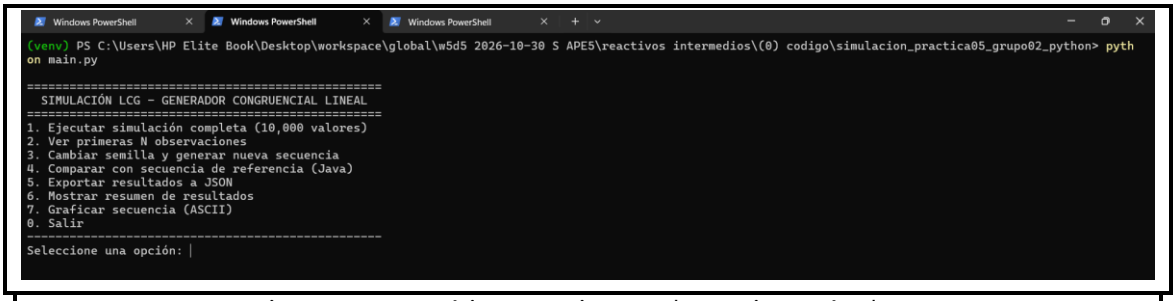


Figura 1 . Menú interactivo Python ejecutándose

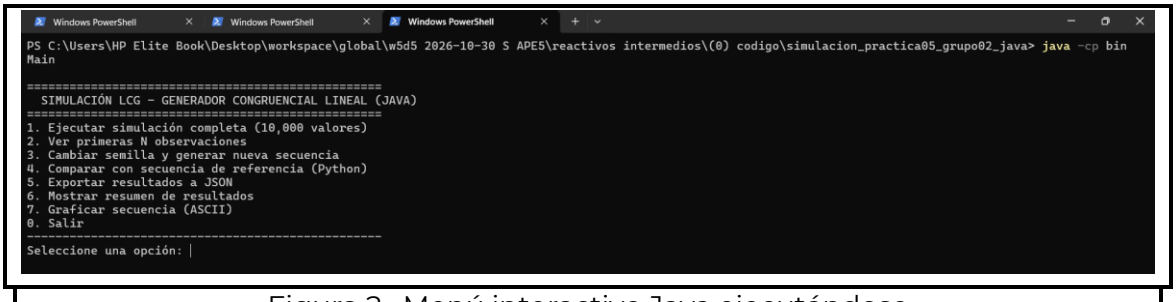


Figura 2 . Menú interactivo Java ejecutándose

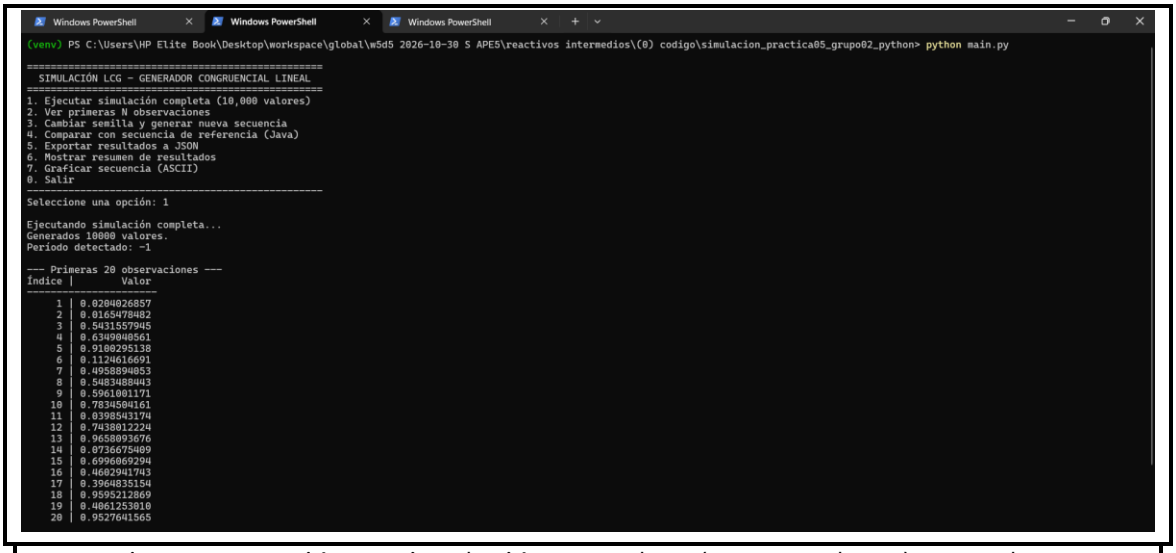


Figura 3 . Opción 1 - Simulación completa (10,000 valores) en Python

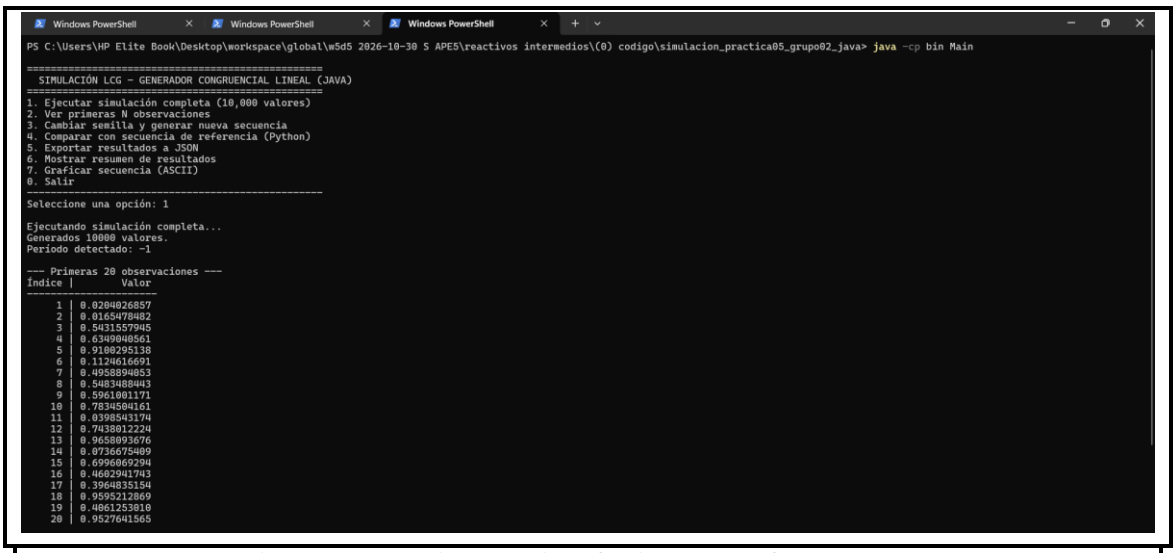


Figura 4 . Opción 1 - Simulación completa en Java

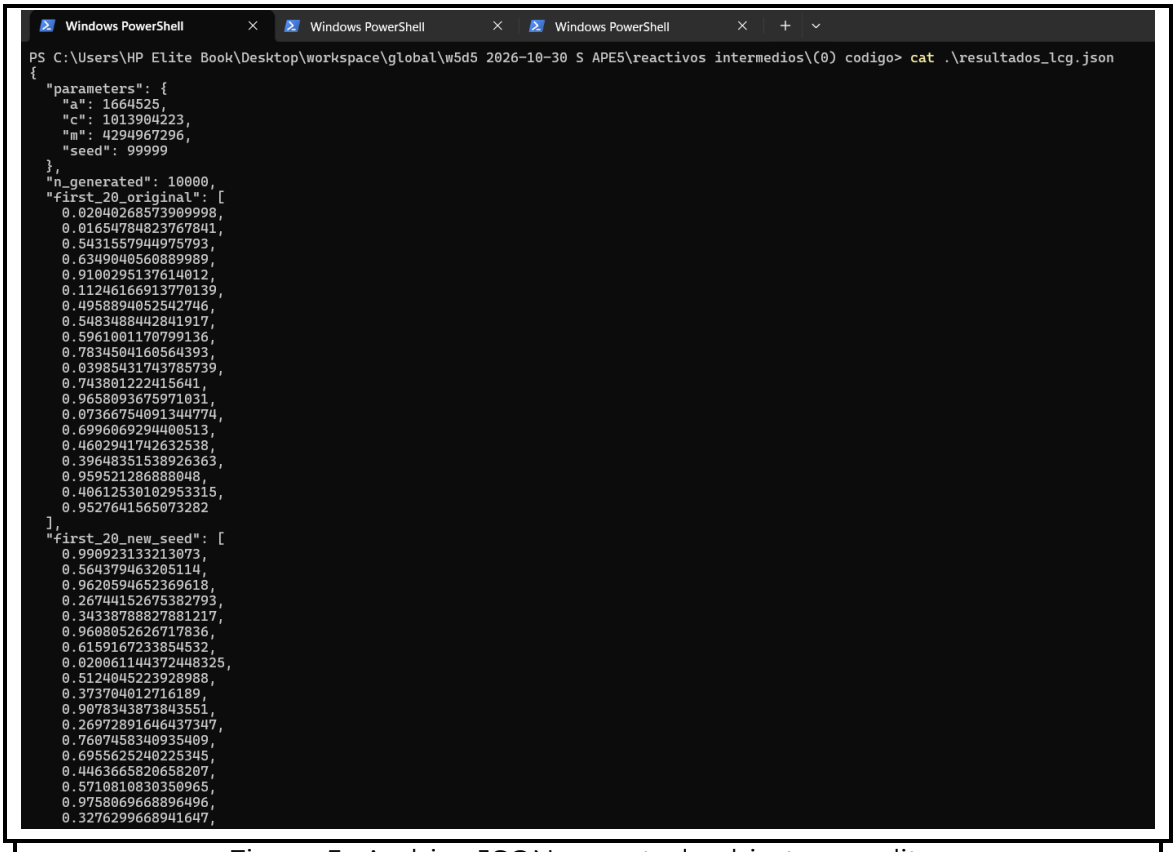


Figura 5 . Archivo JSON exportado abierto en editor